

PDF 3 - LLD Domain Model & Evaluation Architecture

The highest-weight part of the assignment: responsibilities, interfaces, state transitions and extensibility

CipherSchools Hiring Assignment - LLD Practice Platform | September 2026

1. Domain boundary

Keep framework code outside the core. The domain owns practice behavior; adapters own database, AI provider and job execution. This is a modular monolith, not microservices.

2. Core types and responsibilities

Type	Owns	Must not own
Problem	Prompt, requirements, difficulty, rubric reference	User progress or AI calls
Attempt	Learner/problem identity, draft lifecycle, timestamps	Rubric scoring logic
Submission	Immutable snapshot of one attempt	Mutable editor state
Rubric	Ordered criteria, weights, descriptions	Provider-specific prompts
Evaluation	State, result, evaluator version, failure details	Submission mutation
FeedbackItem	criterion, score, evidence, concern, suggestion, confidence	Overall workflow
Evaluator	Evaluate a Submission against Rubric	HTTP/UI concerns
SubmissionValidator	Deterministic completeness/shape checks	Design judgment
EvaluationDispatcher	Schedule evaluation	How a provider reasons

3. Important interfaces

MVP evaluator implementation: GeminiEvaluator implements Evaluator. It calls Google Gemini server-side using the Google AI Studio API key. Domain/application code depends on Evaluator, never directly on the Gemini SDK, so future RuleBasedEvaluator or HumanReviewEvaluator implementations remain additive changes.

```
interface SubmissionPayload {
  format: 'structured-text';
  assumptions: string;
  classes: Array<{ name: string; responsibility: string }>;
  relationships: string;
  mainFlow: string;
  edgeCases: string;
```

```

tradeOffs: string;
}

interface Evaluator {
  readonly kind: string;
  evaluate(input: { submission: Submission; problem: Problem; rubric: Rubric }): Promise<EvaluationResult>;
}

interface SubmissionValidator {
  validate(payload: SubmissionPayload, problem: Problem): ValidationIssue[];
}

interface EvaluationDispatcher {
  dispatch(evaluationId: string): Promise<void>;
}

```

4. State machines

Aggregate	States	Legal transitions
Attempt	DRAFT, SUBMITTED	DRAFT -> SUBMITTED only; submitted snapshot is immutable
Evaluation	QUEUED, EVALUATING, COMPLETED, FAILED	QUEUED -> EVALUATING -> COMPLETED/FAILED; FAILED -> QUEUED on explicit retry

5. Evaluation model

Criterion	Weight	Evaluation
Requirement understanding	15%	AI judgment with cited evidence
Class responsibilities	20%	AI judgment
Coupling/cohesion	15%	AI judgment
Encapsulation/interfaces	15%	AI judgment
Abstraction/patterns	10%	AI judgment; do not reward patterns automatically
Extensibility	10%	AI judgment against change scenarios
Edge cases/testability	10%	Deterministic presence + AI quality
Explanation quality	5%	AI judgment

6. Deterministic vs AI checks

Deterministic	AI
Required sections present and minimum meaningful content	Whether responsibilities are cohesive and well assigned
At least N classes with non-empty responsibilities	Design trade-offs and alternative choices
Valid state transition	SOLID/abstraction analysis grounded in submitted evidence
Duplicate/idempotency detection	Extensibility under changed requirements
Result schema validation	Concrete improvement suggestions

7. Structured AI output contract

Gemini must be prompted with the fixed rubric and required to return structured JSON only. Validate every Gemini response with Zod before persisting or showing it. Invalid/missing criteria must fail the evaluation safely rather than producing partial feedback.

```
{
  "summary": "...",
  "criteria": [{
    "criterionId": "class-responsibilities",
    "score": 0-5,
    "evidence": ["short exact/near-exact evidence from submission"],
    "concern": "...",
    "suggestion": "...",
    "confidence": 0.0-1.0
  }],
  "strengths": [...],
  "nextAttemptFocus": [...]}
}
```

8. Prompt rules that make feedback defensible

- Tell the model explicitly that multiple valid designs exist.
- Give the problem statement, requirements, fixed rubric and learner submission; do not give a single canonical answer as truth.
- Require every criticism to cite evidence or identify missing evidence.
- Do not reward number of classes or number of patterns.
- Score each criterion independently before composing summary.
- Validate structured output with Zod; reject malformed output and mark evaluation Failed.
- Store evaluator name/model, rubric version and prompt version for reproducibility.

9. Change-test proof

Change Test A: text -> class diagram

Add a new SubmissionPayload variant and parser/validator implementation. Attempt, Submission, Evaluation and practice flow remain the same because they depend on a format contract, not text fields directly.

Change Test B: Gemini -> rule-based or human reviewer

Add RuleBasedEvaluator or HumanReviewEvaluator implementing Evaluator. The submission flow and persisted Evaluation aggregate do not change.

10. Patterns justified by change

Pattern	Use	Reason
Strategy	Evaluator implementations	Evaluation method is an explicit

		variation point
Adapter	Google Gemini provider client	Keeps provider SDK out of domain
Repository	Persistence boundary if used	Makes domain tests independent of database
State machine / guarded methods	Attempt & Evaluation transitions	Prevents illegal transitions
Factory - optional	Choose evaluator by config	Only add if more than one implementation exists

Avoid Observer, CQRS, Event Sourcing, Abstract Factory, etc. unless the code genuinely needs them. The guide explicitly warns against patterns added only to show pattern knowledge.